

Better Scheduler

Design Document

A custom Kubernetes scheduler for multi-tenant GPU fairness
with per-namespace guarantees, preemption, and packing

Status	Implemented — in production since v0.1 (Apr 2026); current release v0.7
Version described	v1.32.4-bs-v0.7
Kubernetes base	v1.32.4 (in-tree fork)
Source commit	22dfb9910d3-dirty
Author	Platform / Infrastructure
Date	July 2026
Audience	Any developer; no prior project context assumed

All namespaces, quantities, and node names in this document are illustrative examples.
Live cluster configuration is the source of truth for real values.

TL;DR — the whole design in one page

Better Scheduler is a forked `kube-scheduler` (Kubernetes v1.32.4) that shares a multi-tenant GPU pool fairly: each team (namespace) gets a hard **guarantee** of resources for urgent workloads, everything else runs opportunistically. It is two in-tree scheduler plugins plus decision logging, compiled into one binary.

1. **One PriorityClass defines the guaranteed tier.** A pod with `priorityClassName: guaranteed` may consume resources up to its namespace's configured guarantee. The check runs at scheduling time, per resource: $\text{usage} + \text{request} \leq \text{guarantee}$.
2. **Over guarantee → the pod waits.** It is created normally, stays `Pending` (never rejected), and is re-queued automatically when guaranteed usage in its namespace drops.
3. **Within guarantee but the cluster is full → it preempts normal pods.** Victim selection is namespace-aware: on equal priority, the cost lands first on the namespaces using the most preemptible capacity of the resources the pod lacks.
4. **The freed node is reserved.** After preemption, an in-memory reservation blocks every other pod from the nominated node until the preemptor binds — no one steals the capacity while victims terminate.
5. **Guaranteed GPU pods are packed, not spread.** They can never be evicted, so spreading would fragment the pool permanently; a scoring plugin packs them onto as few nodes as possible.
6. **Normal pods are untouched.** They schedule exactly as with the stock scheduler and remain preemptible.
7. **One process, two profiles.** The binary runs a `better-scheduler` profile (full plugin stack, opt-in via `spec.schedulerName`) and a `default-scheduler` profile that replaces the stock scheduler.
8. **Every decision is explainable.** Per-pod score and rejection logs (gated to the `better-scheduler` profile), lifecycle events with a `decisionID`, config metrics, and a Grafana dashboard.

Three things not to miss

- “Guaranteed” protects against **normal-workload competition only** — not against node failure, maintenance eviction, or admin deletion. Use this wording with users.
- The reservation store is process memory: the reservation filter must be enabled in **every** profile, and the stock `kube-scheduler` must **not** run alongside this one.
- A namespace absent from the configuration has guarantee **zero** — guaranteed pods from unlisted namespaces can never schedule. That is the security backstop, and a common onboarding pitfall.

How to read this document. Evaluating the design? Read this page, the goals (Section 2), Figure 2, and the alternatives comparison (Section 18). Implementing against it? The design sections through the worked example (Section 9). Operating it? Configuration, observability, and release sections. Reviewing robustness? The failure-mode table (Section 15), Section 16, and Section 8.

Contents

1	Overview	4
1.1	What “guaranteed” means — and what it does not	4
2	Goals and Non-Goals	4
2.1	Goals	4
2.2	Non-Goals	4
3	The Two-Tier Model	5
3.1	Why this lives in the scheduler, not in ResourceQuota	5
3.2	Separation of concerns: the admission webhook	5
4	System Architecture	5
4.1	Process and profile topology	6
4.2	Where the plugins hook into the scheduling cycle	7
5	Scheduling Semantics — Walkthroughs	7
5.1	A normal pod	7
5.2	A guaranteed pod, within its guarantee	7
5.3	A guaranteed pod, over its guarantee	7
5.4	A guaranteed pod, within guarantee, but the cluster is full	8
6	Guarantee Accounting Rules	8
7	Namespace-Aware Preemption	9
7.1	Scope	9
7.2	Victim selection and ordering	9
7.3	Preemption events	9
8	Nominated-Node Reservation	10
8.1	The race	10
8.2	The mechanism	10
8.3	End-to-end timeline	10
8.4	Accepted limitations	11
9	A Worked Example	11
10	GPU Packing	12
11	Configuration	13
11.1	Scheduler configuration	13
11.2	Pod opt-in	14
12	Observability	14
12.1	Decision logs	14
12.2	A sample decision trace	14
12.3	Events	16
12.4	Metrics	16
13	Build, Release, Deployment	16
13.1	Test plan	16
14	Performance and Scalability	16
15	Failure Modes — What Happens If... ..	17
16	Security Considerations	18
17	Limitations	18
18	Alternatives: Kueue, Volcano, YuniKorn	19
18.1	Kueue (Kubernetes SIG)	19
18.2	Volcano (CNCF)	19
18.3	Apache YuniKorn	20
18.4	The honest column	20
18.5	Comparison table	20
19	Future Work and Open Questions	21
20	Implementation History	22
21	Appendix: Glossary	22
22	Appendix: Code Map	23

1 Overview

Better Scheduler is a customized `kube-scheduler` built as a fork of Kubernetes v1.32.4. It solves one problem: **fair sharing of an expensive, contended GPU pool between multiple teams**, while keeping idle capacity available for opportunistic use.

The environment it was designed for:

- A shared pool on the order of hundreds of GPUs, spread over multi-GPU nodes (e.g. 8 GPUs per node).
- Multiple teams, each mapped to one or more Kubernetes namespaces.
- Each team needs a **guaranteed** lane for urgent GPU workloads: capacity they can always claim, even when the cluster is full.
- Everything else runs opportunistically and may be displaced when guaranteed work arrives.

What it provides, in one paragraph: a pod that carries a designated PriorityClass (the **guaranteed tier**) may consume resources up to its namespace’s configured guarantee. Within that guarantee it schedules immediately — preempting normal pods if the cluster is full. Beyond the guarantee it simply waits (`Pending`) and is scheduled automatically once its namespace’s usage drops. Normal pods are scheduled exactly as by the stock scheduler. Guaranteed GPU pods are additionally **packed** onto as few nodes as possible, because they can never be evicted and would otherwise fragment the pool permanently.

The design philosophy, fixed at project start and still in force:

- Correctness over optimal packing.
- Simplicity over theoretical fairness — no dominant-resource fairness, no cross-namespace queue arbitration.
- Explicit guarantees over elastic behavior.
- Scheduler stability over feature breadth.

1.1 What “guaranteed” means — and what it does not

The guarantee protects a workload **from competition by normal workloads**, via priority plus quota enforcement. It does **not** make a pod immune to node failure, to eviction for unrelated reasons (node pressure, maintenance), or to administrative deletion. This wording matters when communicating the SLA to users.

Terminology note. The tier was originally called *protected* and the code still uses that word (`protectedPriorityClassName` , “protected pods” in comments and messages). Operationally the concept was renamed to *guaranteed* (a PriorityClass named `guaranteed` , the “guaranteed tier”). In this document the two words are interchangeable.

2 Goals and Non-Goals

2.1 Goals

Six requirements this design must satisfy. The rest of the document refers to them as **R1–R6**: design sections carry a “serves Rn” tag, and the alternatives analysis (Section 18) scores other systems against the same list.

1. **R1 — Scheduling-time namespace guarantee** for one urgent tier: over-cap pods wait as `Pending` , are never rejected, and wake automatically when usage drops.
2. **R2 — Preemption of opportunistic pods** when the cluster is full, with the cost charged to the heaviest borrowers.
3. **R3 — GPU packing** of never-evictable pods, plus closing the nominated-node race so freed capacity is not stolen.
4. **R4 — Zero workload migration**: teams set one PriorityClass; no CRDs, no job wrappers, no queue labels.
5. **R5 — Stock behavior preserved bit-for-bit** for every workload outside the guaranteed tier.
6. **R6 — Minimal operational surface**: no new control-plane components, one config file.

2.2 Non-Goals

Things that could reasonably be goals, but are explicitly chosen not to be:

- **Gang scheduling** (all-or-nothing placement of pod groups). If it becomes a requirement, re-evaluate existing batch schedulers before growing the fork (Section 18).
- **Fairness models beyond the victim-ordering rule** — no dominant-resource fairness, no weighted shares, no cross-namespace queue arbitration.

- **A borrowed / intermediate tier** between guaranteed and normal — no latency SLA or queue semantics required it.
- **MIG-aware guarantees** — `nvidia.com/mig-*` resources never count toward GPU guarantees; MIG policy belongs to the admission webhook.
- **Dynamic policy** — no CRD or informer-driven guarantee reload; configuration is static scheduler args by design.
- **Authenticating who may use the guaranteed PriorityClass** — the admission webhook’s job; the scheduler only provides the zero-guarantee backstop (see Section 16).

3 The Two-Tier Model

	Guaranteed tier	Normal tier
Who	Urgent workloads only	Everything else
Identified by	One designated PriorityClass (exact match on <code>spec.priorityClassName</code>)	Any other / no priority class
Capacity	Hard-capped per namespace by configured guarantees	Opportunistic, unbounded by this system
Can preempt	Yes — normal pods, via native preemption	No
Can be preempted	No (by this system)	Yes
Over the cap	Stays Pending until namespace usage drops	n/a

A third, intermediate “borrowed” tier was considered and deliberately rejected: there was no separate latency SLA and no queueing semantics that would need it, so it would have been complexity without operational benefit.

Guaranteed pods do not preempt each other: they share a single PriorityClass level and preemption only ever targets strictly lower-priority pods.

3.1 Why this lives in the scheduler, not in ResourceQuota

`ResourceQuota` is enforced at **admission time**: pod creation is rejected when the namespace is over its limit. The desired semantics are **scheduling-time**:

- pod creation always succeeds, regardless of current usage;
- an over-guarantee pod stays `Pending` instead of being rejected;
- it is scheduled automatically as soon as namespace usage drops — no client retry loop;
- when the guarantee check passes, the pod continues through the normal scheduling pipeline, including preemption.

Only the scheduler can implement “wait, then preempt” semantics. Hence an in-tree scheduler plugin, compiled into a forked `kube-scheduler` binary — not a webhook, not a scheduler extender (extenders cannot replace victim selection or scoring with the required precision, and add a network hop to every cycle).

3.2 Separation of concerns: the admission webhook

The scheduler deliberately does **not** police who may use the guaranteed PriorityClass. That is the job of a mutating/validating admission webhook (a separate system, outside the fork), responsible for:

- mapping trusted intent (e.g. a label set by tooling) to the guaranteed PriorityClass,
- blocking manual self-assignment of that PriorityClass,
- MIG policy — e.g. forbidding guaranteed MIG workloads. The scheduler counts only exact configured resource names, so `nvidia.com/mig-*` requests never count toward an `nvidia.com/gpu` guarantee.

The scheduler’s only job is enforcing per-namespace guarantees at scheduling time. As a backstop, a namespace absent from the configuration has a guarantee of **zero** — so an unauthorized guaranteed pod cannot schedule at all.

4 System Architecture

The system is the stock `kube-scheduler` plus two custom framework plugins and one instrumentation change, all compiled into a single binary:

Component	Role
NamespaceResourceGuarantee plugin	The core policy: guarantee enforcement (PreFilter), namespace-aware preemption (PostFilter), GPU packing (Score), requeue hints, config metrics. pkg/scheduler/framework/plugins/namespaceresourceguarantee/
NominatedNodeReservation plugin	Reserves a node freed by preemption for the preemptor until it binds, so other pods cannot steal the capacity. pkg/scheduler/framework/plugins/nominatednodereservation/
Decision logging	Per-pod scheduling explainability logs (scores, rejections, counters), emitted only for the better-scheduler profile. pkg/scheduler/schedule_one.go
Config API	NamespaceResourceGuaranteeArgs — internal and v1 types, validation, generated conversions.

Neither plugin is in any default plugin set; both must be enabled explicitly in the scheduler configuration.

4.1 Process and profile topology

► Serves R5 and R6 — see Section 2.

One scheduler **process** runs two **profiles**:

- `better-scheduler` — the profile guaranteed workloads opt into via `spec.schedulerName`. Carries the full plugin stack.
- `default-scheduler` — a profile literally named like the stock scheduler, replacing it. Pods with an empty `schedulerName` land here and get stock behavior, **plus** the reservation filter.

This single-process, two-profile topology is load-bearing, not cosmetic: the reservation store (Section 8) is in-process memory shared across profiles. If the stock kube-scheduler ran as a separate process alongside, it could not see reservations and would race with preemption. Therefore the stock scheduler is replaced entirely, and the reservation filter is enabled in **both** profiles.

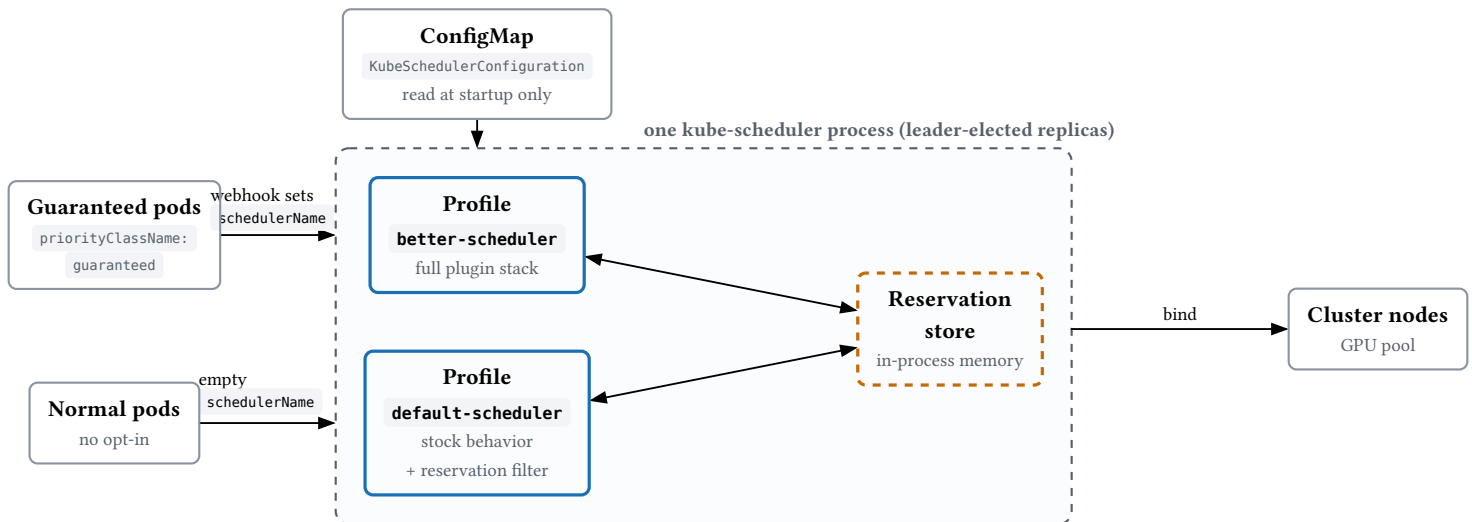


Figure 1: Deployment topology. One binary, two profiles, one shared in-memory reservation store; the stock scheduler is replaced.

4.2 Where the plugins hook into the scheduling cycle

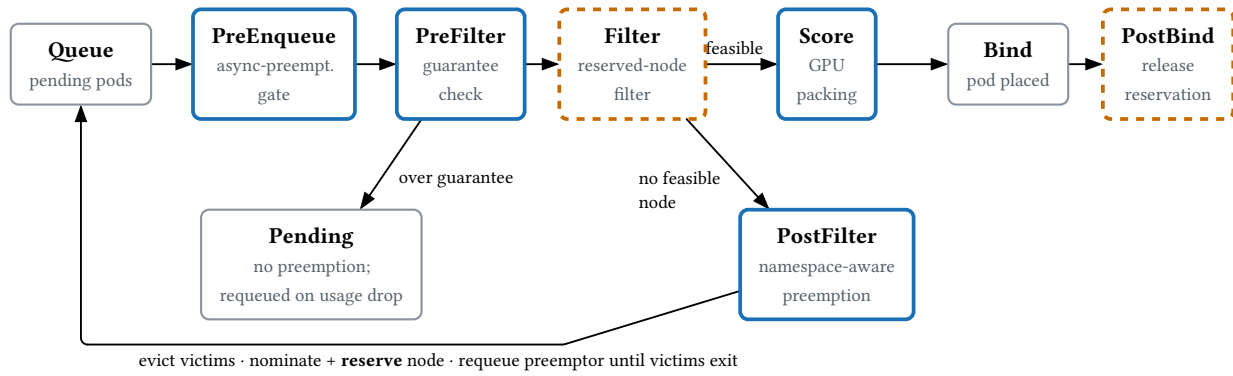


Figure 2: One scheduling cycle. Blue solid = `NamespaceResourceGuarantee` hooks, orange dashed = `NominatedNodeReservation` hooks.

The same hooks as a table:

Phase	Plugin	What happens
PreEnqueue	Guarantee	Holds a pod out of the active queue while its own async preemption is still running.
PreFilter	Guarantee	The guarantee check. Over-guarantee pods are rejected for the cycle as <code>UnschedulableAndUnresolvable</code> .
Filter	Reservation	Rejects every pod except the reservation holder on a reserved node.
PostFilter	Guarantee	Namespace-aware preemption (replaces <code>DefaultPreemption</code> , which is disabled). On success, creates a node reservation.
Score	Guarantee	Packs guaranteed GPU pods onto already-used nodes.
PostBind	Reservation	Releases the reservation once the holder pod binds.
Event queue	Guarantee	<code>EnqueueExtensions</code> : re-queues Pending guaranteed pods when namespace usage may have dropped.

5 Scheduling Semantics — Walkthroughs

Every pod takes one of five paths (Figure 3); the subsections below walk through each.

5.1 A normal pod

The plugins do essentially nothing. The pod is filtered and scored by the stock pipeline. The only additions: it cannot land on a node currently reserved for a preemptor, and (in the `better-scheduler` profile) its decisions are logged.

5.2 A guaranteed pod, within its guarantee

1. `PreFilter` computes the pod's requests for each **configured** resource (cpu, memory, extended scalar resources such as `nvidia.com/gpu`).
2. It computes the namespace's current guaranteed usage from the scheduler's in-memory snapshot: only guaranteed pods, only the same namespace, only pods already assigned to a node.
3. Each configured resource is enforced independently:

$$\text{currentUsage} + \text{requested} \leq \text{namespaceGuarantee}$$

4. The check passes; the pod continues through `Filter` and `Score` like any pod. If nodes have free capacity, it binds. Guaranteed GPU pods are steered by the packing score (Section 10).

5.3 A guaranteed pod, over its guarantee

The `PreFilter` check fails. The pod gets status `UnschedulableAndUnresolvable` with a self-explanatory message:

```
namespace "team-a" protected resource guarantee exceeded:
resource="nvidia.com/gpu" guarantee=32 current=31 requested=2
```

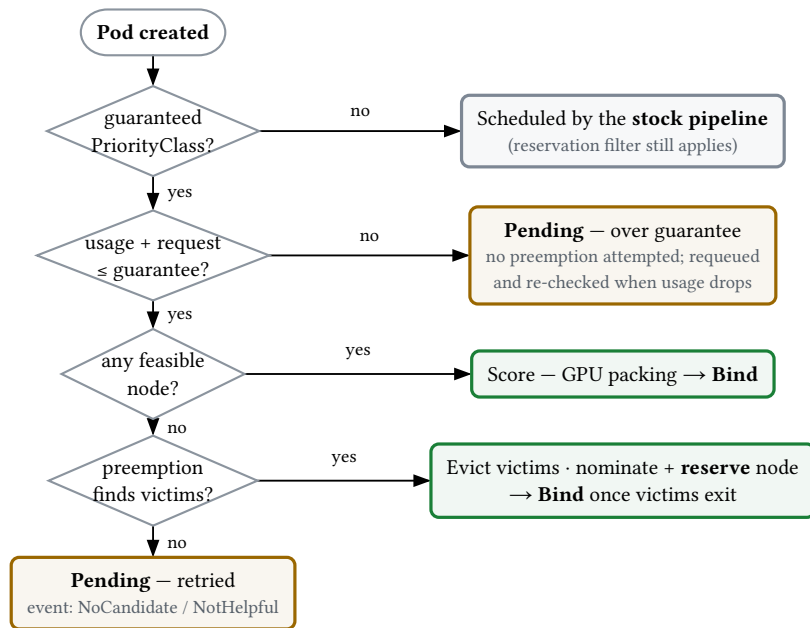


Figure 3: Life of a pod under better-scheduler. Green = scheduled, amber = waiting. A guaranteed pod that requests none of the configured resources follows the stock path unchanged.

`UnschedulableAndUnresolvable` deliberately suppresses preemption for the cycle — being over guarantee is not something evicting other pods can fix. The pod stays `Pending`.

It does not poll. The plugin registers queueing hints for pod `Delete` and `UpdatePodScaleDown` events: when a guaranteed pod in the same namespace is deleted or scales down (or the pending pod itself lowers its request), the pending pod is re-queued and re-evaluated. Irrelevant events don't wake it.

5.4 A guaranteed pod, within guarantee, but the cluster is full

1. PreFilter passes, but Filter finds no feasible node.
2. PostFilter (this plugin's preemption, Section 7) searches candidate nodes for sets of lower-priority victims whose removal would make the pod fit.
3. On success: victims receive deletion (preemption) requests, the pod gets `status.nominatedNodeName`, and the node is **reserved** for it (Section 8).
4. While victims terminate, other pods are filtered away from the reserved node. Scheduling attempts of the preemptor report "waiting" until victims are gone.
5. The pod binds; PostBind releases the reservation.

Failure paths are explicit and observable: preemption may find no candidate node, or determine that preemption cannot help; each outcome is a distinct event on the pod (Section 7.3).

6 Guarantee Accounting Rules

► Serves **R1** — scheduling-time namespace guarantee — see Section 2.

The accounting is intentionally simple and snapshot-based — no API calls, no informer-maintained index; the scheduler's shared snapshot is scanned during the cycle. At the design scale (tens of GPU nodes, a handful of GPU pods per node) this is cheap; an incremental index was rejected as premature optimization.

Counted toward a namespace's usage:

- pods whose `priorityClassName` equals the configured guaranteed class,
- in the same namespace,
- already assigned to a node (`spec.nodeName` set),
- their **requests** for each configured resource.

Ignored: non-guaranteed pods, unscheduled pods, pods in other namespaces, and guaranteed pods requesting zero of every configured resource.

Units: cpu is accounted in **millicores** (so fractional guarantees like `500m` work exactly); memory in bytes; extended resources in whole units (validation enforces integer quantities for them). Micro-example: with `cpu: "2500m"` guaranteed and two guaranteed pods using `1200m + 800m = 2000`, a `600m` pod stays Pending ($2600 > 2500$) while a `500m` pod fits exactly.

A namespace or resource missing from the configuration has guarantee **zero**. This is the security backstop: a guaranteed-priority pod from an unconfigured namespace that requests any configured resource can never schedule.

7 Namespace-Aware Preemption

► Serves **R2** — *preemption charged to the heaviest borrowers* — see Section 2.

`DefaultPreemption` is disabled in the profile; the guarantee plugin implements the scheduler's `PostFilter` extension point and the shared preemption evaluator interface. The mechanics (dry-run candidate nodes, reprieve loop, PDB handling, nominated-node update) mirror the upstream algorithm; the **policy** differs in scope and victim ordering.

7.1 Scope

Preemption runs only for guaranteed pods; for anyone else `PostFilter` returns “preemption is only enabled for protected pods”. Eligibility mirrors upstream: `preemptionPolicy: Never` is respected, and a preemptor whose earlier victims are still terminating on its nominated node waits instead of preempting again.

7.2 Victim selection and ordering

On each candidate node, all pods with strictly lower priority than the preemptor are potential victims. The algorithm removes them, verifies the preemptor would fit, then **reprieves** (re-adds) as many victims as possible, most-important first, keeping the pod feasible. PDB-violating victims are chosen last, exactly as upstream.

What is custom is the importance order. Upstream orders victims by priority, then start time. Better Scheduler inserts a namespace-fairness comparison between the two:

1. **Priority** — higher priority is reprieved first (standard).
2. **Namespace preemptible usage** — on equal priority, compare the two pods' namespaces by their cluster-wide usage from **preemptible** pods (priority below the preemptor) of the resources the node is deficient in. The namespace using **more** preemptible capacity is evicted first.
3. **Own namespace last in ties** — on a further tie, pods in the preemptor's own namespace are evicted before pods of other namespaces.
4. **Start time** — earlier-started pods are reprieved first (standard).

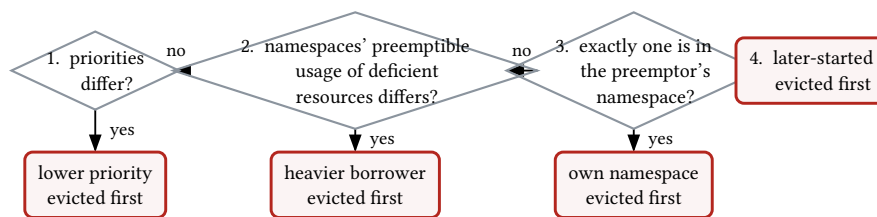


Figure 4: Comparing two potential victims: the first differing rule decides. Rule 2 is why `b-opp-2` loses to `a-opp-1` in the worked example (Section 9).

Deficient resources are ranked by normalized severity (deficit / request), so the comparison focuses on what the incoming pod actually lacks on that node. Namespace usage is computed once per preemption attempt and cached in the decision trace.

The intent: when a guaranteed pod needs room, the cost lands first on the namespaces borrowing the most opportunistic capacity — not on whoever happens to run the newest pods.

7.3 Preemption events

Every preemption decision is traceable. The preemptor receives exactly one classified event per attempt (action `NamespaceResourceGuaranteePostFilter`), and every victim receives one:

Event reason	Type	Meaning
...PreemptionStarted	Normal	Victims selected, node nominated. Note carries <code>decisionID</code> , <code>nominatedNode</code> , and a bounded <code>victims=[...]</code> list.
...PreemptionWaiting	Normal	Waiting for previously preempted pods to finish terminating.
...PreemptionNotHelpful	Normal	Preemption cannot make this pod schedulable.
...PreemptionNoCandidate	Normal	No candidate node found.
...PreemptionError	Warning	Evaluator error.
...Preempted (on victim)	Normal	“Preempted by ns/pod on node X (decisionID=...)”.

(... = `NamespaceResourceGuarantee`.) The `decisionID` is the preemptor’s pod UID, so all events and log lines of one decision correlate. Event notes are bounded to the Kubernetes 1024-character event limit — victim lists are summarized as `pod-1, pod-2, ... (+N more)` with a final truncation guard. This bound exists because unbounded victim lists once caused the API server to reject the events in production.

8 Nominated-Node Reservation

► Serves **R3** — the nominated-node-race half — see Section 2.

8.1 The race

When preemption succeeds, the preemptor only receives `status.nominatedNodeName`. Upstream treats this as a soft hint: **nothing reserves the freed capacity**. Until the victims finish terminating and the preemptor binds, any other pod — in particular one scheduled through the other profile — can be placed onto the node and consume the space. The guaranteed pod then has to preempt again, potentially in a loop, evicting more workloads for nothing.

8.2 The mechanism

A dedicated plugin closes the race with a process-wide, in-memory reservation store:

- After a successful preemption, the guarantee plugin records a reservation: *node X is held for pod P*. On terminal preemption failure it releases P’s reservation.
- The reservation plugin’s **Filter** rejects every pod except the holder on a reserved node (`node reserved for namespace resource guarantee preemption by <ns>/<pod>`). Because it is a Filter, it applies to any profile that enables it — which must be **all** profiles.
- PostBind** releases the reservation when the holder binds (to any node).
- Stale reservations self-clean during Filter: holder deleted, holder UID changed, holder terminating, holder already bound, or holder re-nominated elsewhere. Store semantics: at most one reservation per node and per holder; a re-nomination moves the holder’s reservation; a new holder replaces the old one.
- Reservation lifecycle is visible via events on the holder pod: `NodeReserved`, `NodeReservationReleased`, `NodeReservationCleanedUp` (prefixed `NamespaceResourceGuarantee`).

Figure 5 shows the complete transition set — including the stale-cleanup paths that the timeline below cannot show:

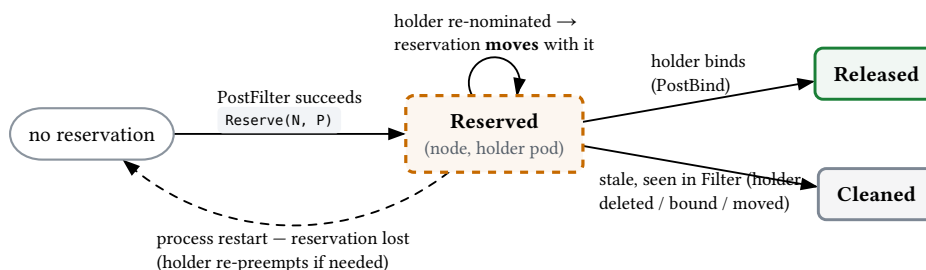


Figure 5: Reservation state machine. All transitions emit an event on the holder pod; the dashed transition is the accepted restart loss (§8.4).

8.3 End-to-end timeline

Figure 6 shows the full preemption-plus-reservation lifecycle, for a guaranteed pod `P`, victims on node `N`, and an unrelated pod `Q` that tries to take the freed capacity.

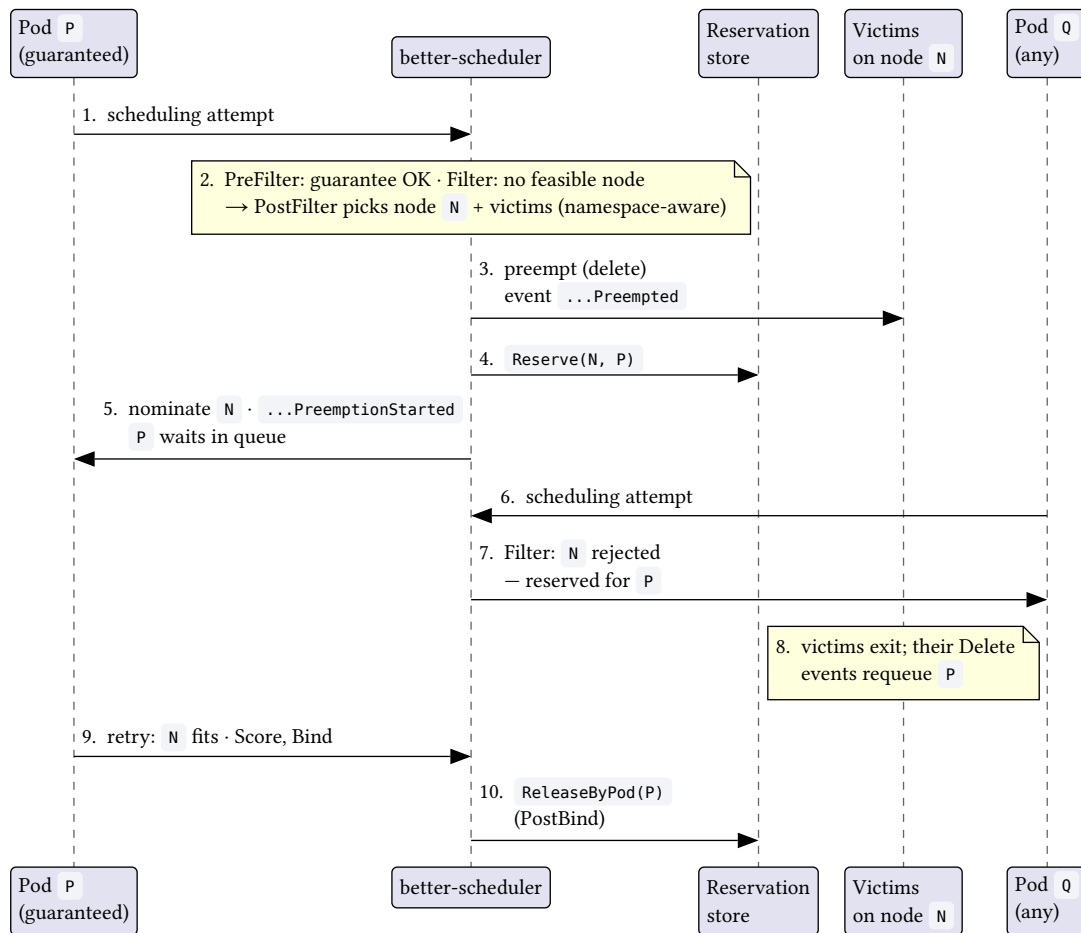


Figure 6: Preemption and reservation lifecycle. Without the reservation (steps 4, 7, 10), `Q` could bind to `N` in step 7 and `P` would have to preempt again.

If `P` retries while victims are still terminating (between steps 5 and 8), the attempt reports `...PreemptionWaiting` rather than preempting again. If `P` is deleted or re-nominated meanwhile, the reservation self-cleans on the next Filter that touches `N`.

8.4 Accepted limitations

The store is process memory. Reservations do not survive a scheduler restart or leader failover, and are invisible to any other scheduler process. Consequences: the filter must be enabled in every profile; the stock scheduler must not run against the same nodes; after a restart mid-preemption, a freed node can be stolen once (the preemptor then simply preempts again). A durable reservation (annotation- or CRD-based) is the known escalation path if this ever proves painful; it has not.

9 A Worked Example

Everything from the last three sections, on one tiny cluster. Two nodes with 8 GPUs each; guarantees configured as `team-a: 8 GPU`, `team-b: 8 GPU` (any other namespace: implicitly 0). Current state:

Pod	Namespace	Tier	GPU	Node
a-train-1	team-a	guaranteed	4	node-1
b-opp-1	team-b	normal	4	node-1
a-opp-1	team-a	normal	2	node-2
b-opp-2	team-b	normal	6	node-2

Derived bookkeeping the plugin sees: **guaranteed** usage `team-a = 4`, `team-b = 0`; **preemptible** (normal) GPU usage `team-a = 2`, `team-b = 10`. Both nodes are full. Figure 7 tracks the cluster state through the events below.

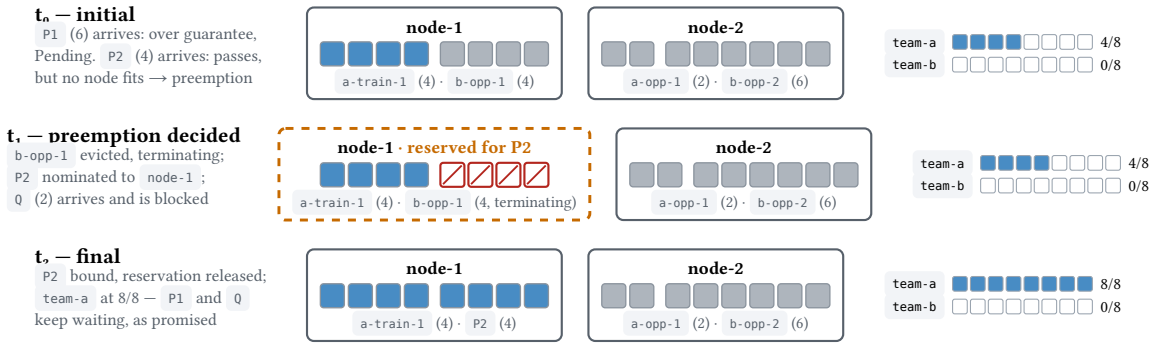


Figure 7: The worked example as cluster states. Blue = guaranteed, gray = normal, red = terminating victim, dashed orange border = reserved node; the right column is each team’s guaranteed usage against its 8-GPU guarantee.

Event 1 — over guarantee. team-a submits guaranteed pod P1 requesting 6 GPU. PreFilter: $4 + 6 = 10 > 8 \rightarrow$ P1 stays Pending with `UnschedulableAndUnresolvable` (“guarantee exceeded: guarantee=8 current=4 requested=6”). No preemption is attempted — evicting pods cannot fix an exceeded guarantee. P1 will be re-queued automatically when team-a’s guaranteed usage drops to 2 or less. (A guaranteed pod from unconfigured team-c would fail the same way forever: guarantee 0.)

Event 2 — within guarantee, cluster full. team-a submits guaranteed pod P2 requesting 4 GPU. PreFilter: $4 + 4 = 8 \leq 8 \rightarrow$ pass. Filter: no node has 4 free GPUs → PostFilter preemption evaluates candidates:

- On node-1 the only lower-priority pod is b-opp-1 (4 GPU) — evicting it frees exactly 4. Victims: { b-opp-1 }.
- On node-2 both normal pods are potential victims. They have equal priority, so the namespace rule decides: team-b’s preemptible GPU usage (10) is higher than team-a’s (2), so b-opp-2 is evicted first and a-opp-1 is reprieved. Evicting b-opp-2 frees $6 \geq 4$ — enough. Victims: { b-opp-2 }.

Both candidates need one victim; the upstream candidate ranking picks the final node (say node-1). Effect of the namespace rule: on either node, the cost lands on team-b — the namespace borrowing the most opportunistic capacity — and never on team-a’s own small opportunistic pod while a heavier borrower is available.

b-opp-1 receives a preemption event and starts terminating; P2 gets `nominatedNodeName: node-1`, the `...PreemptionStarted` event (decisionID = P2’s UID), and node-1 is **reserved** for P2.

Event 3 — the reservation earns its keep. While b-opp-1 terminates, a normal pod Q requesting 2 GPU arrives. Without reservation, Q would bind to node-1 the moment 4 GPUs free up — and P2 would preempt again. With it, Filter rejects node-1 (“reserved for team-a/P2”); node-2 is full; Q waits.

Event 4 — completion. b-opp-1 finishes terminating; its Delete event re-queues P2, which now fits on node-1, is scored, and binds. PostBind releases the reservation; Q is free to compete for whatever capacity remains. Final guaranteed usage for team-a : 8 of 8 — P1 from Event 1 keeps waiting, exactly as the guarantee promises.

10 GPU Packing

► Serves R3 — the packing half — see Section 2.

Guaranteed pods can never be evicted, so fragmentation they cause is **permanent** while they run. With default spreading scores, 1-GPU guaranteed pods land one per node — after which a mostly-free cluster cannot place a single 8-GPU guaranteed pod:

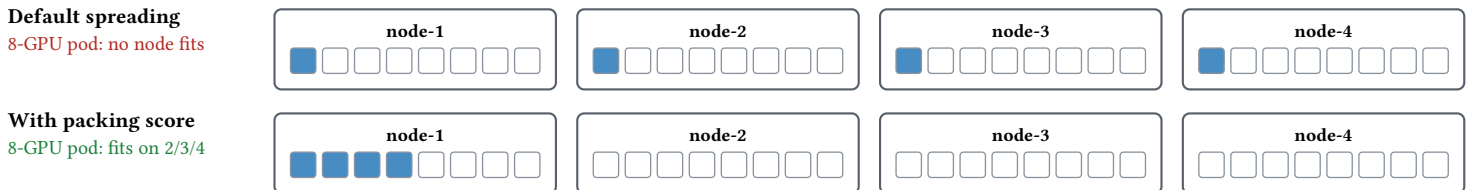


Figure 8: Four 8-GPU nodes, four 1-GPU guaranteed pods (blue). Spread, they block any 8-GPU pod forever — 28 GPUs are free but no node is whole. Packed, three nodes stay whole.

The guarantee plugin therefore implements `Score` for guaranteed pods that request `nvidia.com/gpu` :

$$\text{score} = \min(\text{guaranteedGPUOnNode} + \text{incomingGPU}, \text{allocatableGPU}) \times \frac{100}{\text{allocatableGPU}}$$

- Nodes already carrying guaranteed GPU pods score higher — the plugin packs the guaranteed tier tightly.
- Only **guaranteed** pods' GPU requests count as existing usage. Normal pods' GPU usage is deliberately ignored: normal pods can be preempted away, so they must not attract guaranteed pods.
- Non-guaranteed pods and guaranteed pods without GPU requests score a neutral 0.
- The plugin is enabled with a high weight (e.g. 100) so it dominates the default spreading scores; the profile typically also configures `NodeResourcesFit` with a `RequestedToCapacityRatio` most-allocated shape on `nvidia.com/gpu` so **normal** GPU pods bin-pack too.

Packing is **preventive only**: it stops new fragmentation but cannot repair existing fragmentation — that drains only as pods churn. Deliberately scoped to `nvidia.com/gpu`; `cpu/memory` packing was considered and rejected.

11 Configuration

► Serves *R4* and *R6* — see Section 2.

11.1 Scheduler configuration

The scheduler is deployed as a Deployment (multiple replicas, leader election) with a `KubeSchedulerConfiguration` in a ConfigMap. The config is read **at startup only** — every change requires a rollout. Representative example (values illustrative):

```
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
leaderElection:
  leaderElect: true
  resourceName: better-scheduler
  resourceNamespace: better-scheduler
clientConnection:
  qps: 100          # scheduler default is 50/100; raise when event bursts
  burst: 200       # cause client-side throttling
profiles:
- schedulerName: better-scheduler
  plugins:
    preEnqueue:
      enabled: [{name: NamespaceResourceGuarantee}]
    preFilter:
      enabled: [{name: NamespaceResourceGuarantee}]
    filter:
      enabled: [{name: NominatedNodeReservation}]
    postFilter:
      disabled: [{name: DefaultPreemption}] # replaced by the plugin
      enabled: [{name: NamespaceResourceGuarantee}]
    score:
      enabled: [{name: NamespaceResourceGuarantee, weight: 100}]
    postBind:
      enabled: [{name: NominatedNodeReservation}]
  pluginConfig:
  - name: NamespaceResourceGuarantee
    args:
      apiVersion: kubescheduler.config.k8s.io/v1
      kind: NamespaceResourceGuaranteeArgs
      protectedPriorityClassName: guaranteed
      namespaceGuarantees:
        team-a: {cpu: "64", memory: "256Gi", nvidia.com/gpu: "32"}
        team-b: {cpu: "48", nvidia.com/gpu: "24"}
  - schedulerName: default-scheduler # replaces the stock scheduler
  plugins:
```

```
filter:
  enabled: [{name: NominatedNodeReservation}] # required here too
postBind:
  enabled: [{name: NominatedNodeReservation}]
```

Validation (enforced at startup; the scheduler refuses to start otherwise): the priority class name must be non-empty; at least one namespace with at least one resource guarantee; resources restricted to `cpu`, `memory`, and extended scalar resources; quantities non-negative; extended resources must be integers.

11.2 Pod opt-in

```
spec:
  schedulerName: better-scheduler
  priorityClassName: guaranteed
```

The PriorityClass is ordinary (e.g. `value: 1000000`, `preemptionPolicy: PreemptLowerPriority`). In practice a mutating webhook sets `schedulerName` automatically for pods carrying the guaranteed PriorityClass, so users set only one field — and cannot set it without authorization.

12 Observability

Three signal sources; all correlate on pod identity. Preemption events and decision logs share `decisionID`, and decision logs additionally carry `attempt` to isolate one scheduling retry.

12.1 Decision logs

Upstream kube-scheduler reveals per-node scores and rejection reasons only at verbosity 10, which is process-wide and unusable in production. This fork instead emits full decision detail at plain `Info` level, **gated to the better-scheduler profile only** — always on for exactly the workloads that need explaining, invisible for everything else. Six structured (logfmt-parseable) lines, each carrying `profile`, `decisionID`, `attempt`, and `pod`:

Message	Content
Plugin scored node for pod	per plugin × node: <code>plugin</code> , <code>node</code> , <code>score</code>
Extender scored node for pod	per extender × node
Calculated node's final score for pod	per node: <code>final score</code>
Rejected node for pod	per rejected node: <code>phase</code> (PreFilter / Filter / Extender), <code>plugin</code> , <code>reason</code> , <code>status</code> , <code>synthetic</code>
Scheduling rejection reason summary for pod	rejection reasons aggregated with node counts
Scheduling decision summary for pod	per attempt: <code>cluster_nodes</code> , <code>prefilter_candidate_nodes</code> , <code>prefilter_pruned_nodes</code> , <code>evaluated_candidate_nodes</code> , <code>unevaluated_candidate_nodes</code> , <code>filter_rejected_nodes</code> , <code>extender_rejected_nodes</code> , <code>feasible_nodes</code> , <code>scored_nodes</code>

One honesty detail: the scheduler stops filtering once it has found enough feasible nodes, so nodes divide into *prefilter-pruned*, *explicitly rejected*, and simply *unevaluated* — the counters report these separately instead of calling everything “infeasible”. PreFilter prunes are flagged `synthetic=true` because their per-node lines are synthesized from one aggregate result. `scored_nodes` is 0 when only one node was feasible (scoring is skipped).

A Grafana dashboard (“Better Scheduler Decision Explainability”, Loki + Prometheus) turns these logs into per-pod score charts, rejection breakdowns, and a decision trace: enter a namespace and pod name, see why it landed where it did. Derived charts filter to the latest `attempt`; the raw trace intentionally keeps all attempts in range.

12.2 A sample decision trace

Every line has the same anatomy — four groups of fields (Figure 9):

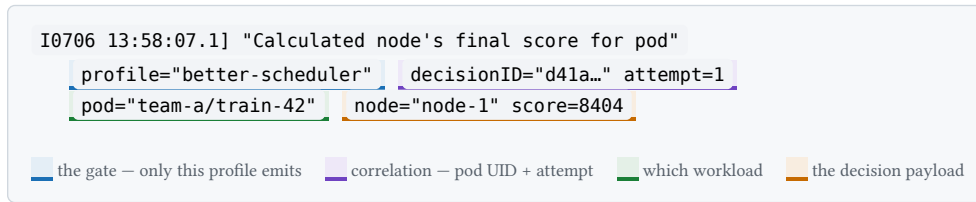


Figure 9: Anatomy of a decision log line. `decisionID` correlates logs with preemption events; `attempt` isolates one scheduling cycle when a pod retries.

What full traces look like (values illustrative, long lines wrapped; `decisionID` / `attempt` shown on the first line of each record and elided after). First, the ordinary case — a guaranteed 2-GPU pod with two feasible nodes; the packing score pulls it toward `node-1`, which already holds guaranteed GPU pods:

```
I0706 13:58:07.1] "Plugin scored node for pod" profile="better-scheduler"
  decisionID="d41a..." attempt=1 pod="team-a/train-42"
  plugin="NamespaceResourceGuarantee" node="node-1" score=75
I0706 13:58:07.1] "Plugin scored node for pod" ...
  plugin="NamespaceResourceGuarantee" node="node-2" score=25
I0706 13:58:07.1] "Plugin scored node for pod" ...
  plugin="NodeResourcesFit" node="node-1" score=9
I0706 13:58:07.1] "Calculated node's final score for pod" ... node="node-1" score=8404
I0706 13:58:07.1] "Calculated node's final score for pod" ... node="node-2" score=3167
I0706 13:58:07.1] "Scheduling decision summary for pod" profile="better-scheduler"
  decisionID="d41a..." attempt=1 pod="team-a/train-42"
  cluster_nodes=4 prefilter_candidate_nodes=4
  prefilter_pruned_nodes=0 evaluated_candidate_nodes=4 unevaluated_candidate_nodes=0
  filter_rejected_nodes=2 extender_rejected_nodes=0 feasible_nodes=2 scored_nodes=2
```

And the preemption case — pod `P2` from the worked example (Section 9): the cluster is full, every node is rejected, then preemption produces events keyed by `decisionID`:

```
I0706 14:02:11.4] "Rejected node for pod" profile="better-scheduler"
  decisionID="8f2c..." attempt=3 pod="team-a/p2"
  phase="Filter" node="node-1" status="Unschedulable" plugin="NodeResourcesFit"
  reason="Insufficient nvidia.com/gpu" synthetic=false
I0706 14:02:11.4] "Rejected node for pod" ... phase="Filter" node="node-2"
  status="Unschedulable" plugin="NodeResourcesFit"
  reason="Insufficient nvidia.com/gpu" synthetic=false
I0706 14:02:11.4] "Scheduling rejection reason summary for pod" ...
  phase="Filter" status="Unschedulable" plugin="NodeResourcesFit"
  reason="Insufficient nvidia.com/gpu" nodes=2
I0706 14:02:11.4] "Scheduling decision summary for pod" profile="better-scheduler"
  decisionID="8f2c..." attempt=3 pod="team-a/p2"
  cluster_nodes=2 prefilter_candidate_nodes=2 prefilter_pruned_nodes=0
  evaluated_candidate_nodes=2 unevaluated_candidate_nodes=0 filter_rejected_nodes=2
  extender_rejected_nodes=0 feasible_nodes=0 scored_nodes=0
```

Events (`kubectl describe pod p2`):

```
Normal NamespaceResourceGuaranteePreemptionStarted better-scheduler
  decisionID=8f2c... phase=started nominatedNode=node-1 victims=[team-b/b-opp-1]
  note="preemption initiated; victim termination may still be in progress"
```

Events (`kubectl describe pod b-opp-1`):

```
Normal NamespaceResourceGuaranteePreempted better-scheduler
  Preempted by team-a/p2 on node node-1 (decisionID=8f2c...)
```

Note the same `decisionID="8f2c..."` on the logs and both events — one identifier ties the whole decision together. When `b-opp-1` finishes terminating, `P2` 's next cycle logs `attempt=4 ... feasible_nodes=1 scored_nodes=0` (a single feasible node skips scoring) and binds.

12.3 Events

See Section 7.3 for the preemption event set and Section 8 for reservation events. Notes are bounded to the 1024-character event limit.

12.4 Metrics

The plugin exports its effective configuration, so dashboards need no ConfigMap access:

- `scheduler_namespace_resource_guarantee_quota{profile, namespace, resource, unit}` — configured guarantee (`unit : millicore / byte / unit`).
- `scheduler_namespace_resource_guarantee_protected_priority_class_info{profile, priority_class}` — constant 1.

Standard scheduler health metrics (`scheduler_schedule_attempts_total` , `scheduler_pending_pods` , plugin/extension-point latency histograms) carry a `profile` label and complete the picture. There is deliberately **no** per-decision metric — per-pod × per-node label cardinality would explode; decisions live in logs.

13 Build, Release, Deployment

- The fork tracks one upstream Kubernetes version (currently v1.32.4); the scheduler binary version must match the cluster.
- Releases are cut by a script (`hack/release-better-scheduler.sh`): preflight tests → image build (amd64) → push to a private registry → annotated git tag carrying the immutable image digest → append to a CSV release ledger. Release naming: `v<k8s-version>-bs-v<subversion>` , e.g. `v1.32.4-bs-v0.7` .
- Deployment is a plain image bump on the scheduler Deployment. There is intentionally no auto-deploy in the release flow.
- Known cosmetic quirk: the binary's self-reported `version=` string embeds the **previous** tag plus a commit offset (the release tag is created after the build, and the build derives its version via `git describe`). The image tag, git tag, and ledger digest are always correct.

13.1 Test plan

- **Unit tests** cover every custom mechanism: the guarantee plugin (PreFilter math, victim selection and ordering, preemption events and note bounding, packing Score, queueing hints), the reservation plugin and store (reserve/release/staleness), args validation, and the decision logs (present for the `better-scheduler` profile, absent for others):

```
go test ./pkg/scheduler/framework/plugins/namespaceresourceguarantee/...
go test ./pkg/scheduler/framework/plugins/nominatednodesreservation/...
go test ./pkg/scheduler/apis/config/validation -run
TestValidateNamespaceResourceGuaranteeArgs
go test ./pkg/scheduler -run 'TestPrioritizeNodesDecisionLogs|
TestFindNodesThatFitPodDecisionLogs'
```

- **Release preflight**: the release script runs the scheduler test packages before building and refuses to release on failure.
- **Cluster smoke tests** (manual, per rollout): scheduler health; a guaranteed pod preempting a normal pod; an over-guarantee pod staying `Pending` with the guarantee-exceeded message; namespace isolation (one team's guarantee not consumable by another).
- **Known gap**: no automated end-to-end suite exercises the fork in CI, and upstream's scheduler e2e is not run against the diff. Accepted while the diff stays small and unit-covered; revisit if the fork grows (Section 19).

14 Performance and Scalability

Everything runs on the scheduling path against the in-memory snapshot — no API calls, no background loops, no incremental indexes. Costs at a glance:

Mechanism	Cost	When it runs
Guarantee check (PreFilter)	One scan of scheduled pods in the snapshot	Per cycle, guaranteed pods only
Preemption (PostFilter)	Dry-run over at most max(10% of nodes, 100) candidates, plus one snapshot scan for namespace preemptible usage	Per failed cycle of a guaranteed pod
Packing Score	O(pods on node) per feasible node	Per cycle, guaranteed GPU pods only
Reservation Filter	One map lookup per node; store holds at most one entry per node	Every pod, every cycle
Decision logs	O(nodes) Info lines per attempt	Every better-scheduler -profile attempt
Config metrics	O(namespaces × resources) gauges	Once, at plugin construction

At the design scale (~50 GPU nodes, a few hundred GPU pods) all of this is negligible — the `scheduler_framework_extension_point_duration_seconds` histograms stay indistinguishable from stock.

At 10× scale, watch these in order:

1. **Decision-log volume** — the only cost that grows as nodes × scheduling churn (roughly 2–4 lines per node per failed attempt of a better-scheduler -profile pod). Mitigations if log-storage cost bites: keep only the summary lines, sample the per-node rejection lines, or put them behind a config knob.
2. **Snapshot scans** (guarantee check, preemptible usage) — linear in pods. Move to an incremental index only when the PreFilter/PostFilter latency histograms say so, not before.
3. **Client QPS** — event bursts during large preemptions; `clientConnection` tuning guidance lives in the configuration section.

15 Failure Modes — What Happens If...

Robustness questions, answered in one place. The recurring theme: every mechanism degrades to stock Kubernetes behavior plus at most one wasted preemption round — never to a stuck cluster.

If this happens...	...then
One scheduler replica crashes / leader failover	Another replica takes leadership and schedules from a fresh snapshot. In-memory reservations are lost: a node freed by an in-flight preemption can be taken once by another pod; the preemptor simply preempts again. Guarantee accounting is unaffected (recomputed from the snapshot every cycle).
All scheduler replicas are down	No pods schedule at all — this binary is the cluster's scheduler (both profiles). Same blast radius as losing the stock kube-scheduler; running pods are unaffected.
Scheduler restarts mid-preemption	Victims keep terminating (deletion was already issued). The preemptor still has <code>nominatedNodeName</code> and is re-queued; upstream nominated-node handling plus a possible second preemption converge. At most one round of duplicate eviction.
The admission webhook is down	Depends on the webhook's <code>failurePolicy</code> , not on this scheduler. Worst case, guaranteed pods are created without <code>schedulerName</code> routing and schedule as normal pods — the guarantee cap still holds for anything that does carry the <code>PriorityClass</code> and reach the profile (and unlisted namespaces are still capped at zero).
The config has a typo	Args validation fails and the new scheduler process refuses to start. With a rolling update, old replicas keep running the old config — the rollout wedges loudly instead of half-applying. Watch rollouts (see the ops docs).

If this happens...	...then
A guarantee is lowered below current usage	Running pods are never evicted by this plugin. New guaranteed pods in that namespace stay Pending until usage drains below the new cap.
A victim never finishes terminating	The preemptor stays nominated and reports <code>...PreemptionWaiting</code> each attempt. Resolution is the same as any stuck-terminating pod (finalizers, node issues) — operator action; the scheduler does not force-delete.
The preemptor is deleted mid-preemption	Victims already deleted are not resurrected (same as upstream preemption). The reservation self-cleans on the next Filter touching that node; the freed capacity goes to whoever schedules next.
Bind fails (API error, node gone)	Stock retry path: the pod returns to the queue. Its reservation is released only on successful bind or staleness — a transient bind failure does not leak the node to others.
Two guaranteed pods race for the last guarantee headroom	Scheduling cycles are serialized per profile; the second pod sees the first as scheduled/assumed in the snapshot and fails the guarantee check cleanly.
The cluster is upgraded before the fork is rebased	Unsupported skew, same as running a mismatched stock scheduler. The fork must be rebased and re-released for the new Kubernetes minor first (see Section 18 for the cost discussion).

16 Security Considerations

- **Priority self-assignment** is defended in depth. First line: the admission webhook maps trusted intent to the guaranteed PriorityClass and blocks manual use. Second line (this scheduler): a namespace absent from `namespaceGuarantees` has guarantee **zero**, so a rogue guaranteed pod from an unauthorized namespace can never schedule; an authorized namespace is still bounded by its own cap.
- **Cross-tenant disruption is by design, but bounded and audited.** A guaranteed pod can evict other teams' normal pods — that is the product. The blast radius is capped by the preemptor namespace's guarantee, victim ordering charges the heaviest borrowers first, and every eviction leaves an audit trail: `...Preempted` events on victims and `...PreemptionStarted` on the preemptor, all keyed by `decisionID`.
- **No new attack surface.** No new API objects, CRDs, webhooks, or controllers; the binary runs with the stock kube-scheduler service account and RBAC. The only added inputs are the static scheduler args (validated at startup) and the PriorityClass name comparison.
- **Information exposure via decision logs.** The logs name pods and namespaces of **all** workloads evaluated by the `better-scheduler` profile. This is the same information stock `-v=10` would emit; the difference is it is always on for that profile. Anyone with read access to the scheduler's logs (or the Loki tenant holding them) can enumerate cluster-wide workload names — scope log-store access accordingly.
- **Events carry no unbounded data.** Preemption notes are truncated to the 1024-character event limit with victim lists summarized, so a pathological victim set cannot be used to stuff events.

17 Limitations

Emergent constraints of the implemented design (deliberate scope exclusions live in the Non-Goals list, Section 2):

- **Guarantee ≠ instant placement.** A guarantee is a share of capacity, not a placement SLO; a large pod can wait due to fragmentation. Packing prevents new fragmentation; it cannot repair existing fragmentation.
- **Config changes require a rollout.** Guarantees are scheduler args read at startup: every change means editing the ConfigMap and rolling the Deployment.
- **In-memory reservations.** Not durable across restarts, invisible to other processes (see Section 8).
- **Priority use is not authenticated here.** Without the admission webhook, anyone can claim the guaranteed PriorityClass — bounded only by the zero-guarantee backstop (see Section 16).

18 Alternatives: Kueue, Volcano, YuniKorn

“Why didn’t you use an existing batch scheduler?” is the right first question. This section compares the three serious candidates against the goals **R1–R6** defined in Section 2, honestly — including the one dimension where every alternative beats a fork. Capability descriptions reflect the projects as of mid-2026; re-verify before re-evaluating.

The systems act at different points of a pod’s lifecycle (Figure 10): Kueue gates **admission** and hands placement to the stock scheduler; Volcano and YuniKorn bring their **own scheduling cores**; Better Scheduler patches the **upstream** core in place. This is why Kueue can compose with this design while the other two compete with it.

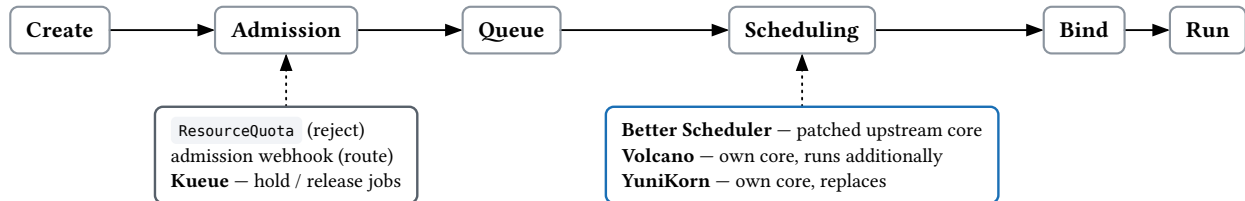


Figure 10: Where each system intervenes in the pod lifecycle. The two-scheduler race arises whenever two independent cores (dotted) place pods onto the same nodes.

18.1 Kueue (Kubernetes SIG)

What it is. A job-queueing **controller**, not a scheduler. Workloads enter `LocalQueue`s backed by `ClusterQueue`s with nominal quotas; Kueue holds them unadmitted (via `spec.suspend` on Jobs, or pod scheduling gates) until quota is available, then releases them to the stock kube-scheduler. Cohorts let queues borrow unused quota from each other, with reclaim (“preempt to get back nominal quota”) and fair sharing; hierarchical cohorts are GA as of early 2026.

Where it shines. This is the philosophically closest system: “held, not rejected” admission, nominal quotas per team, borrowing as the opportunistic mechanism, reclaim as preemption. If the requirement set grew toward job-level fair sharing, all-or-nothing admission, or multi-cluster queueing (MultiKueue), Kueue is the right tool.

Why it doesn’t fit alone. Kueue acts entirely **before** scheduling and delegates placement to kube-scheduler. That misses R3 twice: it cannot pack GPU pods and cannot fix the nominated-node race — both are node-level placement problems. R4 fails too: the borrowing model only works if the **opportunistic** workloads are also enrolled in queues (otherwise there is nothing to reclaim from), which means migrating every simulation, service, and ad-hoc pod in the cluster into queue-labeled, suspendable form. And quota is tracked at **admission** time against queue bookkeeping, not against the scheduler’s live view — a second accounting system to keep honest.

Worth knowing: Kueue **composes** with this scheduler. If job-granular queueing is ever needed, Kueue can gate admission while better-scheduler keeps enforcing scheduling-time guarantees and placement underneath. They are not mutually exclusive.

18.2 Volcano (CNCF)

What it is. A full batch scheduler that runs as an **additional** scheduler alongside the default one. Workloads opt in via `schedulerName: volcano` and gain `PodGroup` / `vcjob` semantics: gang scheduling, queue capacity with proportional/DRF fair share, preempt/reclaim/backfill actions, and a bin-packing plugin. Strong ecosystem integration (Spark, Ray, PyTorch operators).

Where it shines. Gang scheduling and job-level batch semantics — genuinely hard problems this fork deliberately avoids (non-goals).

Why it doesn’t fit. Two schedulers making placement decisions over one node pool is exactly the race condition v0.4 exists to eliminate — Volcano-routed pods and default-scheduler pods would fight over freed capacity with no shared reservation state, so honoring R3 would require migrating **all** workloads to Volcano (violating R4 and R5: its scheduling core is not the upstream framework, so every workload’s behavior changes). Its queue model speaks proportional shares and weights, not “hard per-namespace cap for one priority class with unlimited opportunistic underneath” — expressible, but as an approximation. And it is a large moving system (scheduler + controllers + CRDs + webhook) with its own release cadence, versus a ~4,300-line diff on the upstream scheduler (R6).

18.3 Apache YuniKorn

What it is. A resource scheduler that **replaces** kube-scheduler’s scheduling core entirely (deployed with an admission controller that routes all pods to it). Hierarchical queues carry `guaranteed` and `max` resources; placement rules map namespaces to queues; preemption respects queue guarantees; gang scheduling and bin-packing node-sorting are built in. Rooted in the big-data world (YARN lineage), strong for Spark-style multi-tenancy.

Where it shines. Its `guaranteed / max` queue model is the same **concept** as our namespace guarantees, and it is the operationally closest alternative — it also says “replace the scheduler.”

Why it doesn’t fit. Replacing the scheduling core swaps the semantics of **every pod in the cluster** onto a non-upstream engine (violating R5): upstream scheduler features arrive on YuniKorn’s timetable, subtle behaviors (topology spread, nominated-node handling, plugin scoring) differ, and debugging placement means learning a second scheduler. Our two-tier semantics — hard cap for one PriorityClass, genuinely unlimited opportunistic tier below it — maps only approximately onto queue `guaranteed / max` plus fences. The fork instead changes upstream behavior **only** inside a small, reviewable diff: everything else is bit-for-bit kube-scheduler v1.32.4.

18.4 The honest column

Every alternative wins on one dimension: **nobody has to rebase them onto each Kubernetes release**. The fork’s standing cost is porting a ~4,300-line diff to every upstream minor we adopt and re-releasing (see the release ledger). That cost was accepted knowingly, because it buys R1–R6 exactly rather than approximately. The break-even flips if the requirements grow toward gang scheduling or hierarchical fair share — at that point, revisit YuniKorn (replacement) or Kueue-on-top (composition) before growing the fork.

18.5 Comparison table

	Better Scheduler	Kueue	Volcano	YuniKorn
Kind	Patched kube-scheduler (fork, single binary)	Queueing controller + CRDs beside stock scheduler	Additional scheduler + controllers + CRDs	Replacement scheduler + admission shim
Enforcement point	Scheduling time (PreFilter)	Admission time (suspend / scheduling gates)	Scheduling time (own core)	Scheduling time (own core)
Quota model	Hard per-namespace cap per resource, one urgent tier	ClusterQueue nominal quota; cohorts, borrowing, fair share	Queue capacity, proportional / DRF share	Hierarchical queues with <code>guaranteed / max</code>
Over-cap behavior	Pod Pending, event-driven re-queue	Workload held unadmitted in queue	Job waits in queue	App/pod waits in queue
Opportunistic tier	Native normal pods, zero enrollment	Must also be enrolled in queues (else nothing to reclaim)	Must be routed to Volcano	All pods pass through it by design
Preemption	Native priority + namespace-fair victim ordering	Reclaim / priority at workload granularity	Preempt / reclaim actions, job-aware	Fair-share, respects queue guarantees
Node placement, GPU packing	Custom Score packs guaranteed GPU pods; reservation closes nominated-node race	None — delegates to kube-scheduler	Bin-packing plugin	Bin-packing node sorting
Gang scheduling	No (non-goal)	No (all-or-nothing admission only)	Yes	Yes

	Better Scheduler	Kueue	Volcano	YuniKorn
Workload changes	One PriorityClass	Queue label + suspendable / gated workloads	<code>schedulerName</code> + PodGroup / vcjob CRDs	None per workload (admission routes everything)
Stock behavior for other pods	Bit-for-bit upstream outside the diff	Yes (stock scheduler places pods)	Only for pods not routed to it — which reintroduces the two-scheduler race	No — different core for all pods
Ops footprint	One image + one ConfigMap	Controller, web-hooks, CRDs	Scheduler, controllers, CRDs, web-hook	Scheduler, shim, queue config
Upgrade burden	Rebase fork per Kubernetes minor — the main cost	Independent add-on	Independent add-on	Independent add-on
Decision observability	Per-pod decision logs + Grafana dashboard (Section 12)	Workload events / conditions	Job / PodGroup events	Queue REST API + web UI
Governance	Private fork	Kubernetes SIG	CNCF	Apache

Bottom line: Kueue solves a different layer (admission queueing) and can compose with this scheduler later; Volcano and YuniKorn solve overlapping problems but only by putting **all** workloads on a non-upstream scheduling core; none of the three provides scheduling-time per-namespace caps for a single priority tier, namespace-fair victim selection, and nominated-node reservation — which is precisely the shape of R1–R3.

19 Future Work and Open Questions

Known escalation paths, each with the condition that would make it worth doing. None is planned work today.

Item	What	Revisit when
Durable reservations	Back the in-memory store with an annotation or CRD so reservations survive restarts and failovers.	Restarts mid-preemption observably cause repeated duplicate evictions.
Version-string fix	Create the git tag before building, or inject the release version via ldflags, so the binary self-reports the correct version.	Next time the release script is touched — purely mechanical.
cpu / memory packing	Extend the packing Score beyond <code>nvidia.com/gpu</code> .	Non-GPU fragmentation demonstrably blocks guaranteed cpu/memory pods.
Kueue composition	Run Kueue as an admission-queueing layer on top of this scheduler.	Job-level fair sharing, all-or-nothing admission, or queueing across teams becomes a requirement.
Dynamic guarantee config	CRD- or informer-driven guarantee reload instead of ConfigMap + rollout.	Guarantee edits become frequent enough that rollouts are the bottleneck.
Gang scheduling	Not by growing this fork — re-evaluate YuniKorn/Volcano (Section 18).	All-or-nothing multi-pod placement becomes a requirement.
Decision-log scaling	Sampling or a config knob for per-node rejection lines.	Log-storage cost grows past comfort at larger node counts.

Open question with no answer yet: **upstream rebase cadence**. Each adopted Kubernetes minor costs a rebase of the diff plus a full re-release and re-validation. Skipping minors reduces work but grows each rebase. No policy has been needed so far (the fork still tracks v1.32); one will be, eventually.

20 Implementation History

How the design got here — each release was driven by an observed need, two of them by production incidents (marked ▲ in Figure 11). Full narrative lives in the repository’s documentation tree (docs/better-scheduler/history.md).

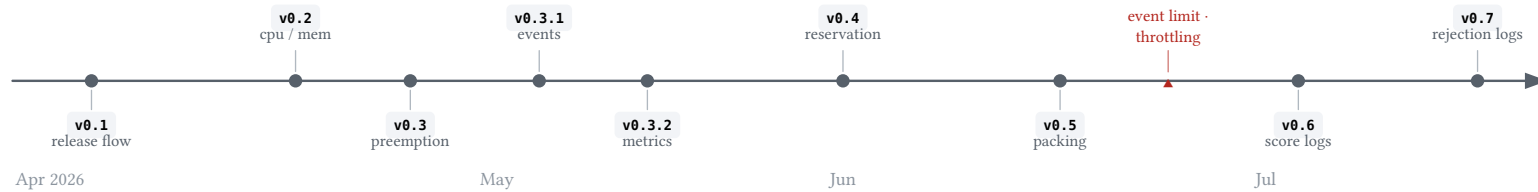


Figure 11: Release timeline. Incidents (▲): oversized preemption events rejected by the API server; client-side throttling under event bursts.

Version	Date	Driver → change
v0.1	2026-04	Needed repeatable self-releases → release script, tag scheme, ledger. Plugin was PreFilter-only, GPU-only.
v0.2	2026-05	Teams needed cpu/memory guarantees too → namespaceGuarantees ResourceList schema, millicore accounting.
v0.3	2026-05	Stock preemption picked victims blind to namespace fairness → custom PostFilter with namespace-aware victim ordering.
v0.3.1	2026-05	Preemption decisions were opaque → classified lifecycle events with decisionID .
v0.3.2	2026-05	Effective config invisible to dashboards → config gauges.
v0.4	2026-06	Observed race: freed nodes stolen while victims terminated → NominatedNodeReservation plugin; two-profile single-process topology.
v0.5	2026-07	Observed fragmentation: spread 1-GPU guaranteed pods permanently blocked 8-GPU jobs → GPU packing Score.
—	2026-07	Production incident: preemption event with a huge victim list rejected by the API server (1024-char limit) → bounded event notes. Production incident: client-side throttling under event bursts → explicit clientConnection QPS/burst.
v0.6	2026-07	“Why did my pod land there?” unanswerable in production → profile-gated score decision logs + Grafana dashboard.
v0.7	2026-07	“Why was my pod not scheduled?” → rejection and decision-summary logs with honest evaluated/unevaluated accounting.

21 Appendix: Glossary

For developers who know Kubernetes but not scheduler internals. Sorted roughly by how early each term appears.

Term	Meaning here
Guaranteed / protected tier	The same thing (code says “protected”, operations say “guaranteed”): pods carrying the one configured PriorityClass, subject to namespace guarantees and able to preempt normal pods.
Guarantee	A per-namespace, per-resource cap on what guaranteed pods may consume, enforced at scheduling time. Not a reservation: unused guarantee is available to normal pods.
PriorityClass	Standard Kubernetes object giving pods an integer priority; preemption may only evict strictly lower-priority pods.
Scheduler profile	A named plugin configuration inside one kube-scheduler process. Pods select it via spec.schedulerName . This system runs two profiles in one process.
Extension point	A hook in the scheduling framework where plugins run: PreEnqueue, PreFilter, Filter, PostFilter, Score, PostBind, ... (see Figure 2).

Term	Meaning here
PreFilter / Filter	PreFilter runs once per pod per cycle (used for the guarantee check); Filter runs per node (used for the reservation check and stock resource fit).
PostFilter	Runs when no node passed Filter — the preemption hook. <code>DefaultPreemption</code> is the stock implementation; this system replaces it.
Snapshot	The scheduler's in-memory view of nodes and scheduled pods, taken per scheduling cycle. All guarantee accounting reads it — no API calls.
Feasible node	A node that survived PreFilter + Filter for the pod. Only feasible nodes are scored; with a single feasible node, scoring is skipped.
Unevaluated node	A candidate never run through Filter because the scheduler already found enough feasible nodes and stopped searching.
Preemption victim	A lower-priority pod deleted to make room for a preemptor.
Reprieve	During victim selection: re-adding a tentative victim that turns out not to be needed, most-important victims first.
Nominated node	<code>status.nominatedNodeName</code> — the node a preemptor is expected to land on after its victims terminate. Upstream it is only a hint; here it is backed by a reservation.
Reservation (holder)	An in-memory claim that a node is held for one pod (the holder) until it binds; enforced by the reservation plugin's Filter.
Preemptible usage	Per namespace: resource requests of scheduled pods with priority below the preemptor. Drives the namespace-aware victim ordering.
<code>decisionID</code>	The workload pod's UID, carried by every decision log line and every preemption event, so all signals of one decision correlate. <code>attempt</code> (also on every log line) further isolates a single scheduling cycle.
PDB	<code>PodDisruptionBudget</code> — limits voluntary disruptions; victims violating a PDB are chosen last, as upstream.
Extended resource	A non-core scalar resource advertised by a device plugin, e.g. <code>nvidia.com/gpu</code> . Always integer quantities.
MIG	NVIDIA Multi-Instance GPU — GPU slices exposed as separate resources (<code>nvidia.com/mig-*</code>). Out of scope for guarantees; policed by the webhook.

22 Appendix: Code Map

What	Where (repo-relative)
Guarantee plugin (all extension points, preemption, packing)	<code>pkg/scheduler/framework/plugins/namespaceresourceguarantee/namespaceresourceguarantee.go</code>
Guarantee config metrics	<code>.../namespaceresourceguarantee/metrics.go</code>
Reservation plugin (Filter, PostBind, staleness)	<code>pkg/scheduler/framework/plugins/nominatednodereservation/plugin.go</code>
Reservation store	<code>.../nominatednodereservation/store.go</code>
Decision / rejection logging	<code>pkg/scheduler/schedule_one.go</code> (<code>detailedScoreLoggingProfile</code>)
Args types (internal / v1)	<code>pkg/scheduler/apis/config/types_pluginargs.go</code> , <code>staging/src/k8s.io/kube-scheduler/config/v1/types_pluginargs.go</code>
Args validation	<code>pkg/scheduler/apis/config/validation/validation_pluginargs.go</code>
Plugin registration	<code>pkg/scheduler/framework/plugins/registry.go</code> , <code>.../names/names.go</code>

What	Where (repo-relative)
Release script and ledger	hack/release-better-scheduler.sh , releases/better-scheduler/releases.csv
Grafana dashboard	releases/better-scheduler/grafana- decision-dashboard.json
Full documentation tree	docs/better-scheduler/